

NURobotics Firmware Course

Week 2: Analog I/O + Interrupts

Course Overview

- Week 1: Intro to Firmware & Programming
- **Week 2: Analog I/O & Interrupts**
- Week 3: Communication Protocols
- Week 4: Driver Development
- Week 5: State Machines + RTOSes
- Week 6: STM32 Fundamentals
- Week 7: STM32 Peripherals
- Week 8: Communication and RTOS Capstone

Pointers

Memory

- Memory has two main attributes:
 - Address
 - The location of the memory (where the memory lives)
 - Value
 - The data stored at that specific location (what memory lives there)

Address	Value
0x01	0x7F
0x02	0x93
0x03	0xA8
0x04	0x12
0x05	0x70
0x06	0x1B
0x07	0xD6

Pointers

Address	Value
0x1000	0x07
0x1001	
0x1002	
0x1003	
0x1004	
0x1005	
0x1006	

int x = 7;



Integer named x is set to 7

Pointers

Address	Value
0x1000	0x07
0x1001	0x1000
0x1002	
0x1003	
0x1004	
0x1005	
0x1006	

```
int x = 7;  
int *pX = &x;
```



Integer pointer named pX
is set to the address of x

Pointers

Address	Value
0x1000	0x07
0x1001	0x1000
0x1002	
0x1003	
0x1004	
0x1005	
0x1006	

```
int x = 7;  
int *pX = &x;  
int y = *pX;
```

Pointers

Address	Value
0x1000	0x07
0x1001	0x1000
0x1002	0x07
0x1003	
0x1004	
0x1005	
0x1006	

```
int x = 7;  
int *pX = &x;  
int y = *pX;
```



Integer named y is set to
the thing pointed to by pX

Pointers

```
uint8_t x = 0x45;  
  
uint8_t *myPointer = &x;  
uint8_t xValue = *myPointer;  
  
*myPointer = 0xB4;  
// What is the value of x?
```

Registers

Everything is a Register! (Memory Mapped I/O)

Registers are small, fast storage elements within the MCU. Registers usually store a very small amount of data (think 8-bit, 16-bit, 32-bit, 64-bit) and can be addressed directly by the MCU. Many devices these days use memory mapped I/O, but registers are still used (at least in name).

Register Uses:

- Operation value storage ($x = y + z \rightarrow \text{reg0} = \text{reg1} + \text{reg2}$)
- Internal peripheral interfacing
 - Timers
 - Internal interrupts
- External peripheral interfacing
 - GPIO
 - Analog inputs
 - Communication ports
 - External interrupts

Direct Register Control (Blink Example)

By modifying specific points in memory, we can directly control pins!

Normal Blink

```
#include <Arduino.h>

void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

Register Level Blink

```
#include <Arduino.h>

volatile uint8_t *portB = (uint8_t*)0x25;
volatile uint8_t *ddrB = (uint8_t*)0x24;

void setup() {
  *ddrB |= (1 << 5);
}

void loop() {
  *portB |= (1 << 5);
  delay(500);
  *portB &= ~(1 << 5);
  delay(500);
}
```

How Arduino Does It

Implementations of Arduino's pinMode and digitalWrite functions can be found here: https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/wiring_digital.c

```
void pinMode(uint8_t pin, uint8_t mode)
{
    uint8_t bit = digitalPinToBitMask(pin);
    uint8_t port = digitalPinToPort(pin);
    volatile uint8_t *reg, *out;

    if (port == NOT_A_PIN) return;

    // JWS: can I let the optimizer do this?
    reg = portModeRegister(port);
    out = portOutputRegister(port);

    if (mode == INPUT) {
        uint8_t oldSREG = SREG;
        cli();
        *reg &= ~bit;
        *out &= ~bit;
        SREG = oldSREG;
    } else if (mode == INPUT_PULLUP) {
        uint8_t oldSREG = SREG;
        cli();
        *reg &= ~bit;
        *out |= bit;
        SREG = oldSREG;
    } else {
        uint8_t oldSREG = SREG;
        cli();
        *reg |= bit;
        SREG = oldSREG;
    }
}
```

```
void digitalWrite(uint8_t pin, uint8_t val)
{
    uint8_t timer = digitalPinToTimer(pin);
    uint8_t bit = digitalPinToBitMask(pin);
    uint8_t port = digitalPinToPort(pin);
    volatile uint8_t *out;

    if (port == NOT_A_PIN) return;

    // If the pin that support PWM output, we need to turn it off
    // before doing a digital write.
    if (timer != NOT_ON_TIMER) turnOffPWM(timer);

    out = portOutputRegister(port);

    uint8_t oldSREG = SREG;
    cli();

    if (val == LOW) {
        *out &= ~bit;
    } else {
        *out |= bit;
    }

    SREG = oldSREG;
}
```

Registers From ATmega328P Datasheet

14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

1.1.3 Port B (PB7:0) XTAL1/XTAL2/TOSC1/TOSC2

Port B is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port B output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port B pins that are externally pulled low will source current if the pull-up resistors are activated. The Port B pins are tri-stated when a reset condition becomes active, even if the clock is not running.

Depending on the clock selection fuse settings, PB6 can be used as input to the inverting Oscillator amplifier and input to the internal clock operating circuit.

Depending on the clock selection fuse settings, PB7 can be used as output from the inverting Oscillator amplifier.

If the Internal Calibrated RC Oscillator is used as chip clock source, PB7...6 is used as TOSC2...1 input for the Asynchronous Timer/Counter2 if the AS2 bit in ASSR is set.

The various special features of Port B are elaborated in "[Alternate Functions of Port B](#)" on page 91 and "[System Clock and Clock Options](#)" on page 36.

Interrupts

What are Interrupts?

Interrupts interrupts the control flow of a program. When an interrupt is triggered a corresponding interrupt handler is called immediately and executed. There are certain instances where interrupts may be disabled, such as during atomic sections or other interrupts.

Common uses:

- Timers
- GPIO
- Communication triggers

Table 12-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

VectorNo.	Program Address ²⁾	Source	Interrupt Definition
1	0x0000 ¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2_OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1_OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0x0022	SPI_STC	SPI Serial Transfer Complete
19	0x0024	USART_RX	USART Rx Complete
20	0x0026	USART_UDRE	USART, Data Register Empty
21	0x0028	USART_TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE_READY	EEPROM Ready
24	0x002E	ANALOG_COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM_Ready	Store Program Memory Ready

How Do We Use Interrupts With Arduino?

`attachInterrupt(digitalPinToInterrupt(pin), ISR, mode)`

```
#include <Arduino.h>

#define LED_PIN 13
#define BUTTON_PIN 11

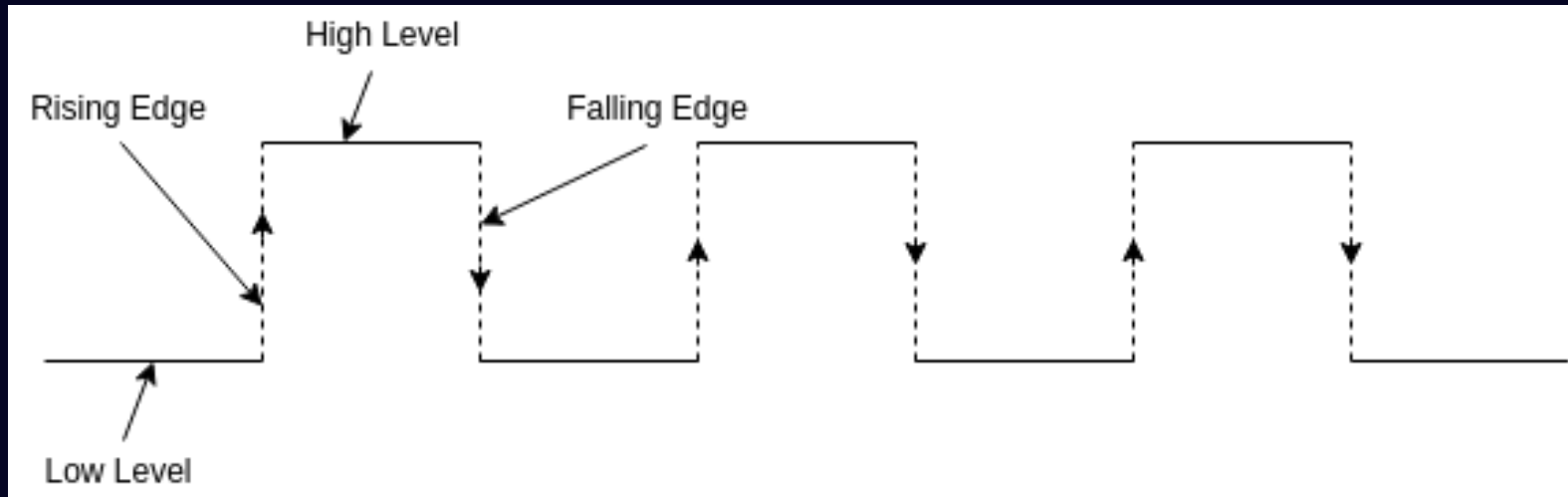
void setup() {
  pinMode(BUTTON_PIN, OUTPUT);

  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), button_isr, RISING);
}

void loop() {
  delay(1); // Nothing needed in the loop!
}

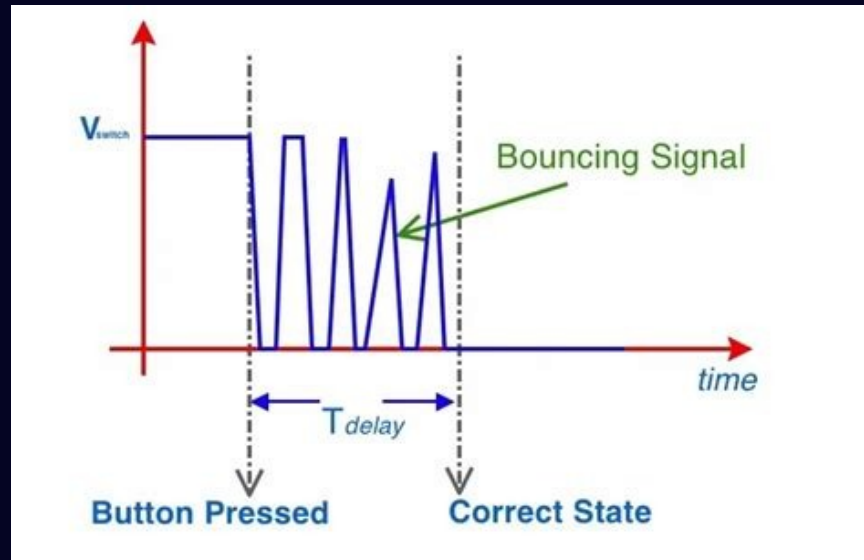
void button_isr() {
  static uint8_t state = LOW;
  if (state == LOW) {
    digitalWrite(LED_PIN, HIGH);
    state = HIGH;
  }
  else {
    digitalWrite(LED_PIN, LOW);
    state = LOW;
  }
}
```

Parts of Digital Signal



Debouncing

A very common use of interrupts is detecting button presses. However, physical buttons have a little bit of natural “bounce”, potentially triggering multiple times before settling. Debouncing removes this unwanted input noise.



```
#include <Arduino.h>

#define LED_PIN 13
#define BUTTON_PIN 11

uint32_t last_trigger = millis();

void setup() {
  pinMode(BUTTON_PIN, OUTPUT);

  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), button_isr, RISING);
}

void loop() {
  delay(1); // Nothing needed in the loop!
}

void button_isr() {
  static uint8_t state = LOW;

  if (millis() - last_trigger < 20) {
    return;
  }

  if (state == LOW) {
    digitalWrite(LED_PIN, HIGH);
    state = HIGH;
  }
  else {
    digitalWrite(LED_PIN, LOW);
    state = LOW;
  }
}
```

Interrupt Memory Safety

Atomic vs Non-Atomic Operations

Atomic actions are completed in a single step and cannot be interrupted. Most actions are non-atomic. However, some actions, like single byte read/writes, can sometimes be atomic, though this is dependent on the architecture the particular microcontroller.

```
uint8_t foo = 45;
uint8_t bar;

// Most 8-bit operations are atomic
bar = foo;
```

```
uint16_t foo = 45;
uint16_t bar;

// 2 byte access is not atomic
bar = foo;
```

This is important because if an ISR is called during the uint16_t read, 'bar' could receive incorrect data

$\text{bar}[7:0] = \text{foo}[7:0] \rightarrow \text{ISR modifies foo} \rightarrow \text{bar}[15:8] = (\text{new})\text{foo}[15:8]$

Volatile Keyword

When a piece of memory is accessed outside of normal operations, either by a second processor or ISR, it is important to mark it “volatile”

```
// the volatile keyword prevents compiler optimizations
volatile bool pressed = false;

void loop() {
    if (pressed) {
        // do something
    }
}

void button_isr() {
    pressed = digitalRead(BUTTON_PIN);
}
```

If “volatile” is not used, the compiler may mark the variable as unchanging and replace it with a fixed value. The “volatile” keyword forces the compiler to access memory each time

Forced Atomicity

While actual atomic actions are naturally too short to be interrupted, interrupts can be disabled to make any process atomic. Sections in which interrupts are disabled can mess with interrupt timing, so these sections should be kept as short as possible.

```
volatile uint32_t count = 0;

void loop() {
    // Disable interrupts to quickly make a copy
    // Just like interrupts, noInterrupt blocks should be quick
    noInterrupts();
    uint32_t count_copy = count;
    interrupts();
}

void button_isr() {
    count++;
}
```

Analog I/O

Analog Input

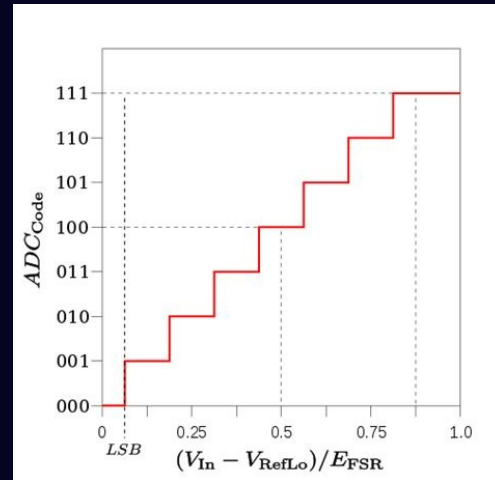
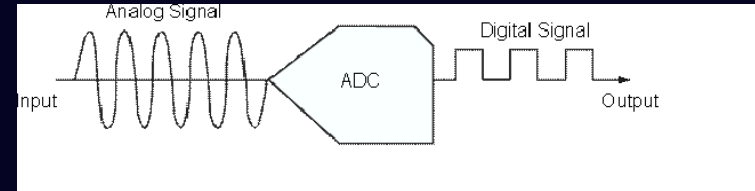
Digital devices cannot interpret analog signals without first converting them into discrete levels. This is what an Analog to Digital Converter (ADC) does.

The Arduino Uno has 6 10-bit ADC on pins A0-A5
These pins convert voltage levels 0-5V to the range of 0-1023 (2^{10})

```
#include <Arduino.h>

void setup() {
  // Analog pins do not need to be initialized manually
}

void loop() {
  // raw is 0-1023
  uint16_t raw = analogRead(A0);
  // voltage is 0.0-5.0V
  float voltage = (raw / 1023.0f) * 5.0f;
}
```



Analog Output

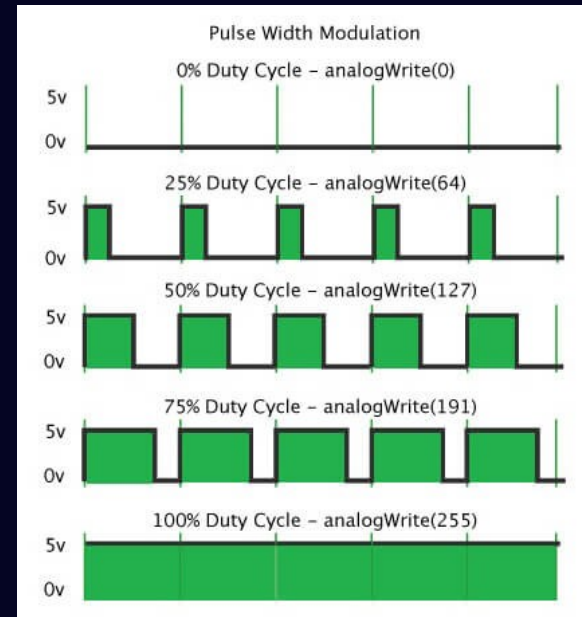
Similar to reading analog signals, digital devices cannot perfectly replicate analog signals. One method of achieving something similar is Pulse Width Modulation (PWM). Instead of outputting 3V, output 5V 60% of the time to achieve the same average voltage!

The Arduino Uno has 6 8-bit PWM capable pins (3, 5, 6, 9, 10, 11)
These pins can have duty cycles of 0-255 (mapped to %)

```
#include <Arduino.h>

void setup() {
  // Analog pins do not need to be initialized manually
}

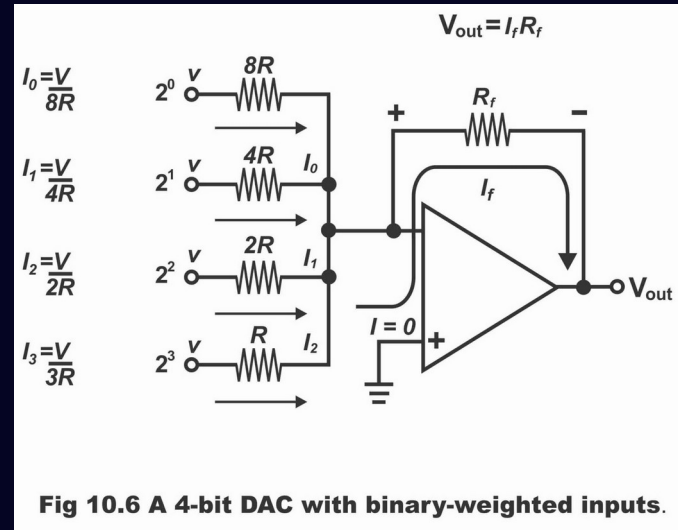
void loop() {
  // 5V 25% of the time ~1.25V
  analogWrite(3, 64);
}
```



Better Analog Output

While PWM can mimic voltage levels, it's not actually changing the voltage. This is where Digital to Analog Converters (DACs) come in. It follows the same logic as ADCs, just reversed

While ADCs are common in microcontrollers, DACs are rarely included.



Lab!

Today's Lab

- Switch Buttons
 - Use interrupts to read button inputs
- PWM
 - Control LED brightness using PWM
- Potentiometer Control
 - Control LED brightness based on potentiometer input